Name: Gargi Devendra Shintre
Div: D15C **Roll No.** 44

Experiment No. 6

Aim:

To apply K-Means and Hierarchical Clustering on sample datasets.

Theory:

Dataset Link:
https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

Dataset Description:
This dataset contains data about customers who visit a mall. The mall wants to understand its customer base to target marketing strategies effectively. This is a classic Unsupervised Learning problem where we do not have labels (e.g., "High Value Customer"); we must discover them.

- **Objective:** To segment customers into distinct groups based on their income and spending habits.
- **Records:** 200 instances.
- **Features:** CustomerID, Gender, Age, Annual Income (k$), Spending Score (1-100).

Variable	Description	Data Type
CustomerID	Unique ID assigned to the customer	Numerical
Gender	Sex of the customer	Categorical
Age	Age of the customer	Numerical
Annual Income (k$)	Annual Income of the customer	Numerical
Spending Score (1-100)	Score assigned by the mall based on customer behavior and purchasing data	Numerical

Theory:

K-Means Clustering

K-Means is a centroid-based algorithm, where we calculate the distance between each data point and a centroid to assign it to a cluster. The goal is to minimize the sum of squared distances between data points and their respective cluster centroids.

Working Principle:

- **Initialize K:** Choose the number of clusters (K).
- **Select Centroids:** Randomly select K data points as initial centroids.
- **Assignment:** Assign each data point to the nearest centroid.
- **Update:** Recalculate the centroids by taking the mean of all points assigned to that cluster.
- **Repeat:** Repeat steps 3 and 4 until centroids do not change (convergence).

Finding Optimal K (Elbow Method)

To determine the best number of clusters, we plot the **Within-Cluster Sum of Squares (WCSS)** against K. The "Elbow Point" represents the K value where adding another cluster does not significantly reduce the variance.

Hierarchical Clustering

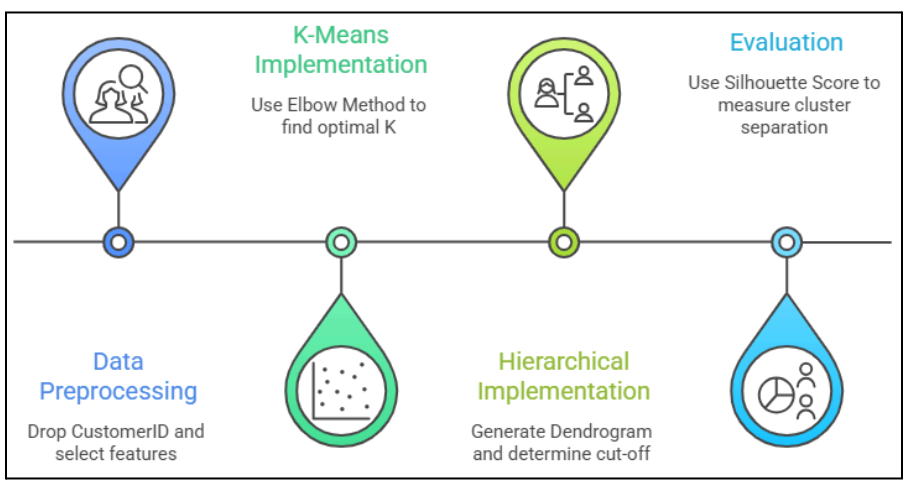
Hierarchical Clustering builds a hierarchy of clusters. We typically use Agglomerative Clustering (Bottom-Up approach), where each data point starts as its own cluster, and pairs of clusters are merged as one moves up the hierarchy.

Visualization (Dendrogram)

A Dendrogram is a tree-like diagram that records the sequences of merges or splits. The optimal number of clusters can be determined by cutting the dendrogram at the longest vertical distance without crossing any horizontal line.

Workflow:

- **Data Preprocessing:**
  - Drop CustomerID as it provides no behavioral information.
  - For 2D Visualization purposes, we primarily focus on Annual Income and Spending Score as features.
- **K-Means Implementation:**
  - Use the **Elbow Method** to find optimal K.
  - Train KMeans model.
  - Visualize the clusters and centroids.
- **Hierarchical Implementation:**
  - Generate a **Dendrogram** using scipy.cluster.hierarchy.
  - Determine the cut-off point for clusters.
  - Train AgglomerativeClustering.
- **Evaluation:** Use the **Silhouette Score** to measure how well-separated the clusters are.



Performance Analysis:

K-Means Results

- **Optimal K:** The Elbow Method plot showed a sharp bend at **K=5**.
- **Cluster Interpretation:** The algorithm successfully identified 5 distinct customer profiles:
    1. **Miser:** High Income, Low Spending.
    2. **General:** Moderate Income, Moderate Spending.
    3. **Target (High Value):** High Income, High Spending.
    4. **Spendthrift:** Low Income, High Spending.
    5. **Careful:** Low Income, Low Spending.

Hierarchical Clustering Results

- **Dendrogram Analysis:** The longest vertical distance suggested **5 clusters** as well, confirming the K-Means finding.
- **Visual Comparison:** The cluster boundaries produced by Hierarchical Clustering were nearly identical to K-Means, validating the stability of these segments.

Algorithm Limitations:

K-Means Clustering

K-Means is prized for its speed, but its simplicity comes with rigid assumptions about the shape and nature of your data.

- **Predefining 'K':** You must specify the number of clusters (k) upfront. If you don't know the underlying structure of your data, you're essentially guessing until you run metrics like the Elbow Method or Silhouette Score.
- **Sensitivity to Outliers:** Since K-Means uses the mean to find the centroid, a single extreme outlier can pull the center of the cluster far away from the actual dense mass of data points.
- **Spherical Assumption:** K-Means assumes clusters are roughly spherical and of similar size. It struggles significantly with elongated, "U-shaped," or complex geometric patterns.
- **Initial Seed Sensitivity:** The final result can change based on where the initial centroids are randomly placed. You might get stuck in a "local optimum" rather than the best global solution.

Hierarchical Clustering

While Hierarchical clustering (especially Agglomerative) is great for seeing how data relates at different levels, it has its own "baggage."

- **Computational Expense:** This is the big one. It has a complexity of roughly O(n^3) or O(n^2 log n). As your dataset grows from thousands to millions of rows, the time and memory required explode, making it unusable for "Big Data."
- **Irreversibility:** Once two clusters are merged (in bottom-up) or split (in top-down), that step cannot be undone. If a mistake is made early in the process, it ripples through the rest of the hierarchy.
- **Sensitivity to Noise:** Like K-Means, noise can result in "chaining," where distinct clusters are erroneously joined together by a thin bridge of outliers.
- **Interpretation Difficulty:** While a **Dendrogram** is helpful for small sets, it becomes a literal "wall of lines" that is impossible to read once you have more than a few hundred data points.

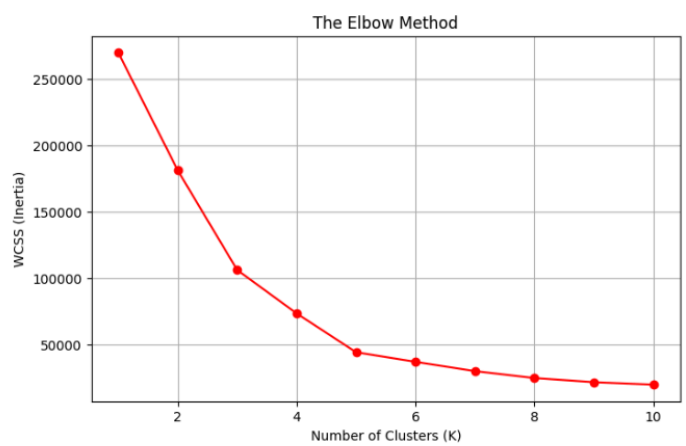
Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans,
AgglomerativeClustering
from sklearn.metrics import
silhouette_score
import scipy.cluster.hierarchy as sch

#
═══════════════════════════════════════════
# 1. LOAD AND PREPROCESS DATA
#
═══════════════════════════════════════════

# 1. Load Dataset
try:
    df = pd.read_csv('Mall_Customers.csv')
    print("Dataset loaded successfully.")
except:
    print("Error: Please upload
'Mall_Customers.csv' to Colab.")

# 2. Feature Selection
# We use Annual Income and Spending Score
for easier 2D visualization
X = df.iloc[:, [3, 4]].values

#
═══════════════════════════════════════════
# 2. K-MEANS CLUSTERING (PART A)
#
═══════════════════════════════════════════
print("\n" + "="*50)
print("PART A: K-MEANS CLUSTERING")
print("="*50)

# 1. Elbow Method to find optimal K
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i,
init='k-means++', random_state=42,
n_init=10)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

# Plot Elbow
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o',
color='red')
plt.title('The Elbow Method')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS (Inertia)')
plt.grid(True)
plt.show()

print("From the graph, the Elbow is at K =
5.")

# 2. Train K-Means with K=5
kmeans = KMeans(n_clusters=5,
init='k-means++', random_state=42,
n_init=10)
y_kmeans = kmeans.fit_predict(X)

# 3. Visualizing K-Means Clusters
plt.figure(figsize=(10, 6))
# Plot each cluster
plt.scatter(X[y_kmeans == 0, 0],
X[y_kmeans == 0, 1], s=80, c='red',
label='Cluster 1 (Careful)')
plt.scatter(X[y_kmeans == 1, 0],
X[y_kmeans == 1, 1], s=80, c='blue',
label='Cluster 2 (Standard)')
plt.scatter(X[y_kmeans == 2, 0],
X[y_kmeans == 2, 1], s=80, c='green',
label='Cluster 3 (Target)')
plt.scatter(X[y_kmeans == 3, 0],
X[y_kmeans == 3, 1], s=80, c='cyan',
label='Cluster 4 (Sensible)')
plt.scatter(X[y_kmeans == 4, 0],
X[y_kmeans == 4, 1], s=80, c='magenta',
label='Cluster 5 (Careless)')
# Plot Centroids
plt.scatter(kmeans.cluster_centers_[:, 0],
kmeans.cluster_centers_[:, 1], s=200,
c='yellow', label='Centroids',
edgecolors='black')

plt.title('K-Means Clustering of
Customers')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()

# 4. Evaluation
score_kmeans = silhouette_score(X,
y_kmeans)
print(f"Silhouette Score (K-Means):
{score_kmeans:.4f}")

#
═══════════════════════════════════════════
# 3. HIERARCHICAL CLUSTERING (PART B)
#
═══════════════════════════════════════════
print("\n" + "="*50)
print("PART B: HIERARCHICAL CLUSTERING")
```

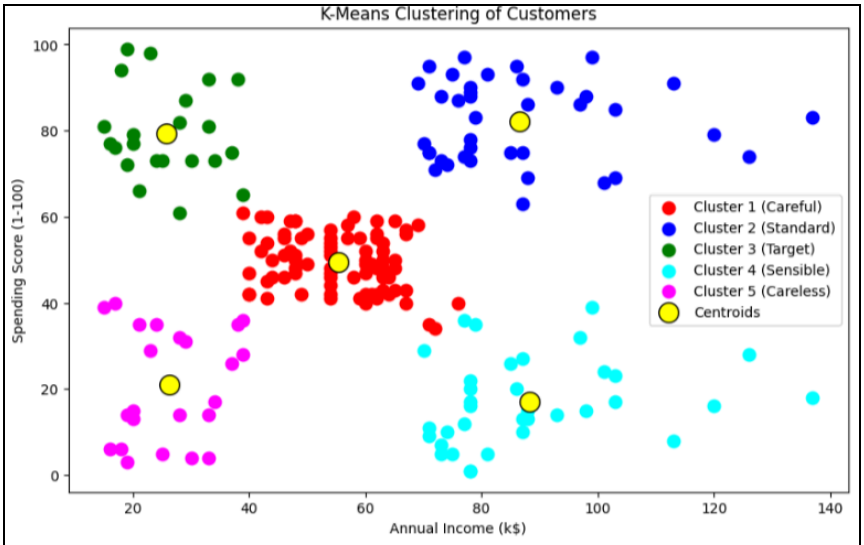
```
print("="*50)

# 1. Dendrogram
plt.figure(figsize=(10, 6))
dendrogram = sch.dendrogram(sch.linkage(X,
method='ward'))
plt.title('Dendrogram')
plt.xlabel('Customers')
plt.ylabel('Euclidean Distance')
plt.show()

print("The longest vertical line without
crossing implies 5 clusters.")

# 2. Train Agglomerative Clustering
hc = AgglomerativeClustering(n_clusters=5,
linkage='ward') # 'ward' minimizes
variance
y_hc = hc.fit_predict(X)

# 3. Visualizing Hierarchical Clusters
plt.figure(figsize=(10, 6))
plt.scatter(X[y_hc == 0, 0], X[y_hc == 0,
1], s=80, c='red', label='Cluster 1')
plt.scatter(X[y_hc == 1, 0], X[y_hc == 1,
1], s=80, c='blue', label='Cluster 2')
plt.scatter(X[y_hc == 2, 0], X[y_hc == 2,
1], s=80, c='green', label='Cluster 3')
plt.scatter(X[y_hc == 3, 0], X[y_hc == 3,
1], s=80, c='cyan', label='Cluster 4')
plt.scatter(X[y_hc == 4, 0], X[y_hc == 4,
1], s=80, c='magenta', label='Cluster 5')

plt.title('Hierarchical Clustering of
Customers')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()

# 4. Evaluation
score_hc = silhouette_score(X, y_hc)
print(f"Silhouette Score (Hierarchical):
{score_hc:.4f}")
```

Output:

```
==================================================
PART A: K-MEANS CLUSTERING
==================================================
```

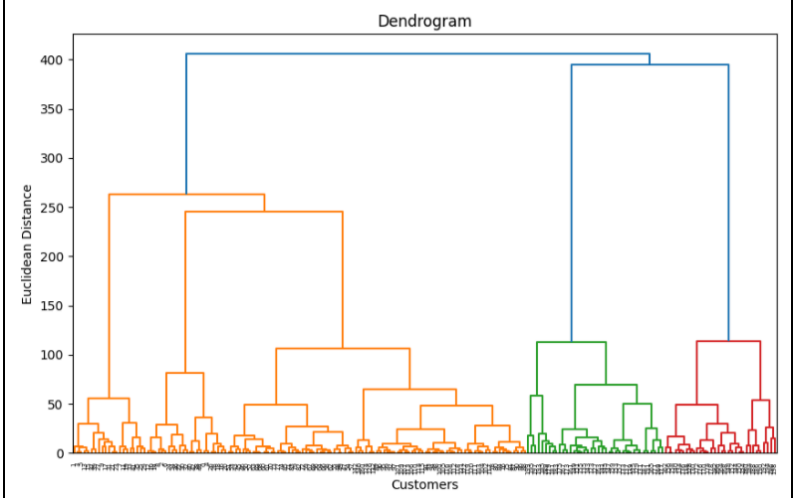



```
From the graph, the Elbow is at K = 5.
```

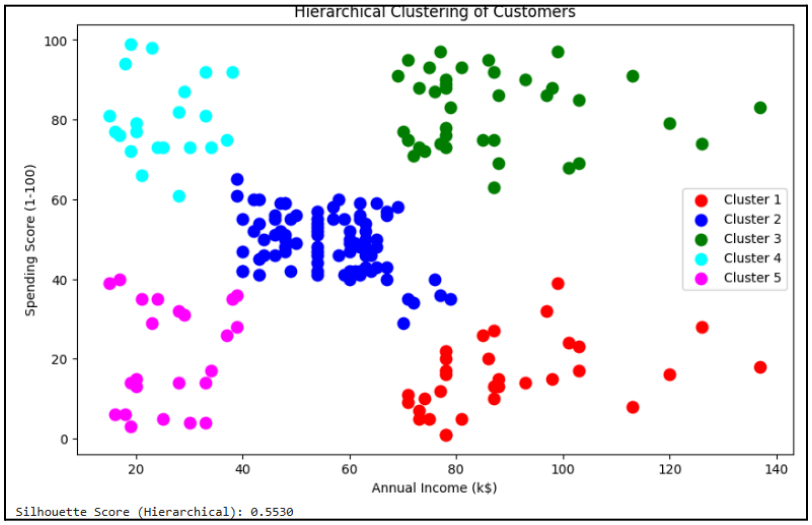



```
Silhouette Score (K-Means): 0.5539
```

```
==================================================
PART B: HIERARCHICAL CLUSTERING
==================================================
```




```
The longest vertical line without crossing implies 5 clusters.
```




```
Silhouette Score (Hierarchical): 0.5530
```

Conclusion:

In this experiment, we applied K-Means and Hierarchical Clustering to segment mall customers. Both algorithms converged on the same result: the customer base consists of 5 distinct behavioral groups. The "High Income, High Spending" group is the prime target for marketing luxury items, while the "Low Income, High Spending" group may be targeted for sales and discounts. This demonstrates how unsupervised learning uncovers actionable business insights without labeled data.